

YModem Batch Analysis – Revision 4 – Xiphod, Jan. 2026

The follow is an annotated log of a YModem Batch transfer between ZOC9 on Windows 11 and TELIX on an MS-DOS '486 based system.

The test files were used, all selected and queued to be sent from ZOC9 to TELIX at 9600 baud.

TEST0.DAT 0 byte

TEST1.DAT 33 bytes

TEST2.DAT 132 bytes (slightly above 128-byte block threshold)

TEST3.DAT 528 bytes (slightly beyond 512-byte threshold)

TEST4.DAT 2112 bytes (slightly beyond 2048-byte threshold)

TEST5.DAT 8448 bytes

Transfer Setup Configuration

REMOTE

(receiver of the file)

486 PC

MS-DOS 6.22

TELIX

SENDER

(uploader of the batch of file)

i7 x86_64

Windows 11

ZOC 9

^ From log perspective, this node is
both the RECEIVER (RX) and
TRANSMITTER (TX)

These systems were connected via a null modem cable, using 8-N-1 at 9600 baud. A USB adapter was used on the i7 SENDER.

In an RS-232 connection, both nodes perform both TX (transmit) and RX (receive) functions. The logging was running on the faster, more performant system – the i7 laptop. So while the SENDER was the node uploading/sending the file, the REMOTE system also sends status responses (so it is not a RECEIVER-ONLY node).

Color Key

PURPLE	Sent by the REMOTE system, received by the SENDER.
BLUE	Sent by the SENDER, received by the REMOTE.
GREEN	Sent by the SENDER, the REMOTE should compute this same value.
RED	String translation of the hex sequence.
ORANGE	Same as BLUE, but unsure if part of official standard.

Begin Transfer Description via Example

All logged entries are in hex. e.g. #43 == 67 decimal == ASCII 'C'

This transfer describes the batch-sequence upload (via YModem) of the **TESTx.DAT** files described earlier. This intentionally includes a blank zero-byte file, then increasingly larger files thereafter. The "RX>" and "TX>" entries are prefixed with a timestamp for reference only (to see the approximate timing between events), they do not represent any data exchange over the wire.

RS-232 SERIAL PORT BYTE STREAM

DESCRIPTION / EXPLANATION

<20260124151805.949 RX>

#43 --> 'C'

Attention to START/CONTINUE

Optional, I typically see this as "CCC". It is sort of like +++, which switches a Hayes modem into command-mode. This rapid "CCC" should get interpreted as switching over into "file transfer mode" (with YModem). The sender can open file-select dialog, or start to send a pre-selected set of files.

At this point, the receiver waits and monitors for a #01 or #02.

And should be prepared to receive an entire header packet of at least 128+3 bytes.

<20260124151809.340 TX>

#01#00#FF

Header block, #01 == 128 byte block

NOTE: If the filename or its path are long (over 128 bytes), this could start with #02 to indicate a 1024-byte block instead.

---START CRC CONTENT[

#54#45#53#54#30#2E#44#41#54#00

"TEST0.DAT" <NUL>

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /

#00#00#00#00#00#00]---END CRC CONTENT (128 bytes)

#00#47

(16-bit CRC)

<20260124151809.486 RX>

Before responding, the receiver verifies that it can create the requested file. This includes path and filename. If file already exist, it might auto choose an alternate name, or ask to overwrite. It could respond NAK or CAN (cancel), or ACK if all ok. The receiver does not yet know how many files, or what sizes.

#06#43

<ACK> 'C' (agree with the CRC, continue)

<20260124151809.608 TX>

#04

<EOT> Nothing left for this stream, all bytes sent.

NOTE: ZOC supports a 'non-standard' EOT exchange. There are two main conventions for this...

OPTION 1: After EOT, just respond with ACK "C". This is actually two steps, the ACK responds to the EOT. Then the "C" is just restarting

the protocol again for the next expected file.
 OPTION 2: EOT (initial), NAK (fake-NAK), EOT (second), ACK "C"
 This is the more correct "True YModem" fashion for responding to EOT.
 A "fake NAK" is sent first, a second EOT is then expected, which
 gets the actual ACK response. Then the "C" continues the protocol.

<20260124151809.616 RX>

#06#43

<ACK> 'C' (acknowledge, continue for next file)

Acknowledges that the file has been written
 (or for 0 bytes, maybe discarded). Asks to
 continue for any subsequent files.

<20260124151810.970 TX>

#01#00#FF

Header block (#01 == 128 byte block)

---START CRC CONTENT[

#54#45#53#54#31#2E#44#41#54#00

"TEST1.DAT" <NUL>

#33#33#20

"33" <SPACE> (bytes)

#31#35#31#32#33#36#31#37#31#33#36#20

"15123617136" <SPACE> (time modified, in octal)

#30#20

"0" <SPACE> (mode)

#30#20

"0" <SPACE> (serial_number)

#35#20

"0" <SPACE> (other)

#31#31#32#35#33

"11253" (??? by ZOC)

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \

\

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \

\

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block

> pad fill the block

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /

/

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /

/

#00#00#00#00#00#00#00#00#00#00#00#00#00#01]---END CRC CONTENT (128 bytes)

(#01 at end = number of 128-byte data blocks)
 (16-bit CRC, up to filename)

#2F#B0

<20260124151811.114 RX>

#06#43

<ACK> 'C' (agree with the CRC, continue)

<20260124151811.239 TX>

#01#01#FE

Data block (#01 == 128 byte block)

---START CRC CONTENT[

#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31 [16 bytes]

#32#33#34#35#36#37#38#39#30#20#01#01#01#01#01 [16 bytes]

#19#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 [1 byte, total of 33 expected]

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \

\

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \

\

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block

> pad fill the block

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /

/

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /

/

]---END CRC CONTENT (128 bytes)

(16-bit CRC)

#03#DC

<20260124151811.376 RX>

#06

<ACK> (agree with the CRC, no retransmit - continue implied)

<20260124151811.376 TX>

#04

<EOT> Nothing left for this stream, all bytes sent.

<20260124151811.392 RX>

#06#43

<ACK> 'C' (acknowledge, continue for next file)

<20260124151812.944 TX>

#01#00#FF

Header block (#01 == 128 byte block)

---START CRC CONTENT[

#54#45#53#54#32#2E#44#41#54#00

"TEST2.DAT" <NUL>

#31#33#32#20

"132" <SPACE> (bytes)

#31#35#31#32#33#36#31#37#34#34#30#20

"15123617440" <SPACE> (time modified, in octal)

```
#30#20 "0"<SPACE> (mode)
#30#20 "0"<SPACE> (serial_number)
#34#20 "0"<SPACE> (other)
#31#31#32#32#30 "1122" (??? by ZOC)
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00]---END CRC CONTENT (128 bytes)
(#02 at end = number of 128-byte data blocks)
(16-bit CRC)
#67#1D
<20260124151813.089 RX>
#06#43 <ACK> 'C' (agree with the CRC, continue)
<20260124151813.212 TX>
#01#01#FE Data block 1 (#01 == 128 byte block)
---START CRC CONTENT[
#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31 [16 bytes]
#32#33#34#35#36#37#38#39#30#20#01#01#01#01#01#01 [16 bytes]
#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19 [16 bytes]
#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01#01 [16 bytes]
#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18 [16 bytes]
#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [16 bytes]
#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17 [16 bytes]
#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01 [16 bytes, 128 bytes so far for file]
]---END CRC CONTENT (128 bytes)
(16-bit CRC)
#07#C1
<20260124151813.356 RX>
#06 <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151813.360 TX>
#01#02#FD Data block 2 (#01 == 128 byte block)
---START CRC CONTENT[
#01#01#01#19#00#00#00#00#00#00#00#00#00#00#00#00 [4 bytes used, reached 132 byte total expected]
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
]---END CRC CONTENT (128 bytes)
(16-bit CRC)
#46#9E
<20260124151813.496 RX>
#06 <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151813.496 TX>
#04 <EOT> Nothing left for this stream, all bytes sent.
<20260124151813.514 RX>
#06#43 <ACK> 'C' (acknowledge, continue for next file)

<20260124151815.096 TX>
#01#00#FF Header block (#01 == 128 byte block)
---START CRC CONTENT[
#54#45#53#54#33#2E#44#41#54#00 "TEST3.DAT" <NUL>
#35#32#38#20 "528" <SPACE> (bytes)
#31#35#31#32#33#36#31#37#35#32#32#20 "15123617522" <SPACE> (time modified, in octal)
#30#20 "0" <SPACE> (mode)
#30#20 "0" <SPACE> (serial_number)
#33#20 "3" <SPACE> (other)
#31#31#30#38#38 "11088" (??? by ZOC)
```

```

#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#05]---END CRC CONTENT (128 bytes)
                                     (#05 at end = number of 128-byte data blocks)
#D2#44                               (16-bit CRC)
<20260124151815.241 RX>
#06#43                               <ACK> 'C' (agree with the CRC, continue)
<20260124151815.361 TX>
#01#01#FE                               Data block 1 (#01 == 128 byte block)
---START CRC CONTENT[
#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31 [16 bytes]
#32#33#34#35#36#37#38#39#30#20#01#01#01#01#01 [16 bytes]
#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19 [16 bytes]
#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01#01 [16 bytes]
#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18 [16 bytes]
#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [16 bytes]
#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17 [16 bytes]
#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01 [16 bytes, 128 bytes so far for file]
]---END CRC CONTENT (128 bytes)
#07#C1                               (16-bit CRC)
<20260124151815.503 RX>
#06                               <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151815.503 TX>
#01#02#FD                               Data block 2 (#01 == 128 byte block)
---START CRC CONTENT[
#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16 [16 bytes]
#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01 [16 bytes]
#01#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12 [16 bytes]
#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01 [16 bytes]
#01#01#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11 [16 bytes]
#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20 [16 bytes]
#01#01#01#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F [16 bytes]
#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30 [16 bytes, 256 bytes so far for file]
]---END CRC CONTENT (128 bytes)
#7D#0C                               (16-bit CRC)
<20260124151815.652 RX>
#06                               <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151815.652 TX>
#01#03#FC                               Data block 3 (#01 == 128 byte block)
---START CRC CONTENT[
#20#01#01#01#01#01#01#19#01#02#04#05#07#0B#0C#0E [16 bytes]
#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39 [16 bytes]
#30#20#01#01#01#01#01#01#19#01#02#04#05#07#0B#0C [16 bytes]
#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38 [16 bytes]
#39#30#20#01#01#01#01#01#01#19#01#02#04#05#07#0B [16 bytes]
#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37 [16 bytes]
#38#39#30#20#01#01#01#01#01#01#19#01#02#04#05#07 [16 bytes]
#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36 [16 bytes, 384 bytes so far for file]
]---END CRC CONTENT (128 bytes)
#33#77                               (16-bit CRC)
<20260124151815.791 RX>
#06                               <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151815.791 TX>
#01#04#FB                               Data block 4 (#01 == 128 byte block)
---START CRC CONTENT[
#37#38#39#30#20#01#01#01#01#01#19#01#02#04#05 [16 bytes]
#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35 [16 bytes]
#36#37#38#39#30#20#01#01#01#01#01#01#19#01#02#04 [16 bytes]

```

```

#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34 [16 bytes]
#35#36#37#38#39#30#20#01#01#01#01#01#01#19#01#02 [16 bytes]
#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33 [16 bytes]
#34#35#36#37#38#39#30#20#01#01#01#01#01#01#19#01 [16 bytes]
#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32 [16 bytes, 512 bytes so far for file]
]---END CRC CONTENT (128 bytes)
#65#AE (16-bit CRC)
<20260124151815.948 RX>
#06 <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151815.948 TX>
#01#05#FA Data block 5 (#01 == 128 byte block)
---START CRC CONTENT[
#33#34#35#36#37#38#39#30#20#01#01#01#01#01#19 [16 bytes, 528 bytes of total file size reached]
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
]---END CRC CONTENT (128 bytes)
#6C#D3 (16-bit CRC)
<20260124151816.091 RX>
#06 <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151816.091 TX>
#04 <EOT> Nothing left for this stream, all bytes sent.
<20260124151816.091 RX>
#06#43 <ACK> 'C' (acknowledge, continue for next file)

<20260124151817.607 TX>
#01#00#FF Header block (#01 == 128 byte block)
---START CRC CONTENT[
#54#45#53#54#34#2E#44#41#54#00 "TEST4.DAT" <NUL>
#32#31#31#32#20 "2112" <SPACE> (bytes)
#31#35#31#32#33#36#31#37#35#37#34#20 "15123617574" <SPACE> (time modified, octal)
#30#20 "0" <SPACE> (mode)
#30#20 "0" <SPACE> (serial_number)
#32#20 "2" <SPACE> (other)
#31#30#35#36#30 "1056" (UNKNOWN by ZOC - can be omitted)
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 \
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 > pad fill the block
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#11 /
]---END CRC CONTENT (128 bytes)
(#11 at end = number of 128-byte data blocks)
#FC#43 (16-bit CRC)
<20260124151817.755 RX>
#06#43 <ACK> 'C' (agree with the CRC, continue)
<20260124151818.405 TX>
#02#01#FE Data block 1 (#02 == 1024 byte block)
---START CRC CONTENT[
#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [32 bytes]
#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [32 bytes]
#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [32 bytes]
#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [32 bytes]
#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [32 bytes]
#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01 [32 bytes]
#01#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20 [32 bytes]
#01#01#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30 [32 bytes]
#20#01#01#01#01#01#19#01#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39 [32 bytes]

```



```
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
#00#00#00#00#00#00#00#00#00#00#00#00#00#00#00 /
]---END CRC CONTENT (128 bytes)
#59#9A                                     (16-bit CRC)
<20260124151820.184 RX>
#06                                         <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151820.184 TX>
#04                                         <EOT> Nothing left for this stream, all bytes sent.
<20260124151820.192 RX>
#06#43                                     <ACK> 'C' (acknowledge, continue for next file)
```

[illegible]

(16-bit CRC)

<ACK> (agree with the CRC, no retransmit - continue implied)

Data block 2 (#02 == 1024 byte block)

[illegible]

(16-bit CRC)

<ACK> (agree with the CRC, no retransmit - continue implied)

Data block 3 (#02 == 1024 byte block)

[illegible]

(16-bit CRC)

<ACK> (agree with the CRC, no retransmit - continue implied)

Data block 4 (#02 == 1024 byte block)

[illegible]

(16-bit CRC)

<ACK> (agree with the CRC, no retransmit - continue implied)

Data block 5 (#02 == 1024 byte block)

[illegible]

```
#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35#36#37#38#39#30#20#01#01#01#01#01#19#01#02#04#05#07
]---END CRC CONTENT (1024 bytes)
```

#02#06#F9 Data block 6 (#02 == 1024 byte block)

```
#B1#69                                     (16-bit CRC)
<20260124151828.776 RX>
#06                                         <ACK> (agree with the CRC, no retransmit - continue implied)
<20260124151829.309 TX>
```

#02#07#F8 Data block 7 (#02 == 1024 byte block)

```
#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34#35
#36#37#38#39#30#20#01#01#01#01#01#01#19#01#02#04
#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33#34
#35#36#37#38#39#30#20#01#01#01#01#01#01#19#01#02
#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32#33
#34#35#36#37#38#39#30#20#01#01#01#01#01#01#19#01
#02#04#05#07#0B#0C#0E#0F#11#12#16#17#18#19#31#32
```

[illegible]

NOTES

For the header blocks, the number-of-blocks at the end of the header block is optional. Note that it assumes the use of 128-byte blocks. This impacts the computed CRC – a valid client/receiver would match the same CRC computation, but would otherwise ignore this piece of data.

ZOC automatically switch to 128-byte blocks when the bytes remaining goes below a certain threshold (about 512-bytes). This is prudent, especially at low baud rates, so that the last block doesn't have to be a full 1KB length. ZOC will also choose to use 128-byte blocks when the transfer rate is low (below 9600 baud).

Header blocks can switch to 1024-byte block if the filename is long (or contains a long path). It remains ambiguous on what constraints the path has – relative, absolute, upper/lower case, and direction of slashes. It is implied that the convention used must be understood and supported by the receiving node.

The “(UNKNOWN by ZOC)” parts of the Block Header is indicated just since it was what was observed in the recorded log. But it can be just “00 ... 00” sequence.

If the filesize is large (255 blocks * 1024 bytes, so over 256KB), it appears that the HEADER MARKER rolls over from FF → 00, and the HEADER INVERSE MARKER rolls from 00 → FF after 256K of transferred data.

NOTE: Normal **XModem** starts the header at “01 FE” and XModem didn't have a 00 block. Some robust file transfer implementations could simultaneously support XModem and YModem, and to do so it would need to look for NAK starter (instead of “C”) to indicate CHECKSUM checking instead of CRC (and then also start at “01 FE” blocks, than would then roll over to “01 FE” accordingly, i.e. FF → 01, 01 → FE).

The very last “all empty” block seems to be the way for the SENDER to indicate that no more additional files are coming, and that the batch xfer is concluded. That is, a blank/null filename and no filelength, CRC results as “00 00.”

Sample C Implementation for 16-bit CRC

```
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>

uint16_t calculate_xmodem_crc(const unsigned char* data, size_t len)
{
    uint16_t crc = 0x0000; // Initial value (other CRC might start with 0xFFFF)
    int16_t signed_ref;
    uint8_t i;

    while (len-->0) {
        // XOR with the current byte shifted into the high byte
        printf("%c(%d) %X ", *data, *data, crc);
        printf(" ^ (%X)", (uint16_t)(*data) << 8);
        crc ^= (uint16_t)(*data++) << 8;
        signed_ref = crc;
        printf("= %X(%d)\n", crc, signed_ref);
        for (i = 0; i < 8; ++i) {
            printf(" %d %X --> ", i, crc);
            if (crc & 0x8000) {
                // If the most significant bit is set, shift left
                // and XOR with the polynomial
                crc = (crc << 1) ^ 0x1021; // The XMODEM polynomial
            } else {
                // Otherwise, just shift left
                crc <<= 1;
            }
            signed_ref = crc;
            printf("%X(%d)\n", crc, signed_ref);
        }
    }

    return crc; // The result is used directly without final XOR
}

int main()
{
    uint16_t crc_result;
    int16_t unsigned_ref;
    char test_str[128] = {
        0x01, 0x02, 0x04, 0x05, 0x07, 0x0B, 0x0C, 0x0E, 0x0F, 0x11, 0x12, 0x16, 0x17, 0x18, 0x19, 0x31,
        0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39, 0x30, 0x20, 0x01, 0x01, 0x01, 0x01, 0x01,
        0x19, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
    };

    crc_result = calculate_xmodem_crc(test_str, 128);
    unsigned_ref = crc_result;
    printf("%X(%d)", crc_result, unsigned_ref);

    return 0;
}
```

Sample BASIC Implementation for 16-bit CRC

Style is X16 BASLOAD, but dialect is very similar to CBM BASIC V2 (just add line numbers).

```
REM XIPHOD - DEC 2025

MAGIC.VALUE% = $1021
REM THESE ONLY WORK WHEN X <= 32767 AND X >= -32768 (SPECIFICALLY THE "OR" AND "AND" KEYWORDS)
REM X XOR Y = (X OR Y) AND NOT (X AND Y)
REM ALTERNATIVE: (X+Y) - 2*(X AND Y)
DEF FN XM(X) = (X + XV) - 2*(X AND XV)

DIM CRC.DATA%(1024) : REM CRC.DATA%(0) CONTAINS LENGTH

GOSUB INIT.CRC.DATA

GOSUB CALC.CRC

GOSUB PRINT.CRC.AS.HEX

END

REM INTERNAL EXAMPLE OF RECEIVING SOME DATA THAT WE WANT TO
REM THEN MAINTAIN A CRC OF. THIS "SIMULATES" A FILE LOAD OR
REM SERIAL DATA RECEIVE.
INIT.CRC.DATA:
  I% = 0
INIT.CRC.NEXT:
  READ A% : REM SIGNED 16-BIT
  IF A% < 0 THEN CRC.DATA%(0) = I% : RETURN
  I% = I% + 1 : CRC.DATA%(I%) = A%
  GOTO INIT.CRC.NEXT

CALC.CRC:
  CRC.RESULT = $0 : REM INITIAL VALUE (XMODEM INIT 0, FOR IBM 3740 - START WITH $FFFF)
  DATA.IDX% = 1 : REM DATA STARTS AT INDEX 1

NEXT.BYTE:
  IF DATA.IDX% > CRC.DATA%(0) THEN RETURN : REM CRC.RESULT COMPUTED

  REM CHR$(80) ENABLES A "DON'T INTERPRET" SIGNAL, SO PETSCII SYMBOL
  REM OF SUBSEQUENT OUTPUT IS SHOWN INSTEAD OF CRT-EFFECT APPLIED.
  PRINT CHR$(80);CHR$(CRC.DATA%(DATA.IDX%));"(";CRC.DATA%(DATA.IDX%);") ";CRC.RESULT;" ";

  XV = CRC.DATA%(DATA.IDX%) * 256 : REM (CRC.DATA AT CURRENT INDEX) << 8
  REM THE FOLLOWING IS NECESSARY SO WE CAN USE "XV" LATER IN THE DO.XOR SUBROUTINE
  IF XV > 32767 THEN XV = XV - 65536
  IF XV < -32768 THEN XV = XV + 65536

  CRC.TEMP = CRC.RESULT
  CRC.TEMP = FN XM(CRC.TEMP) : REM XOR'D WITH XV
  CRC.RESULT = CRC.TEMP
  GOSUB PRINT.CRC.AS.HEX : PRINT : REM PRINT HEX$(CRC.RESULT)

  XV = MAGIC.VALUE% : REM PREPARE THAT WE WILL XOR BY THE XMODEM POLYNOMIAL
  FOR I = 0 TO 7

    REM CAN'T USE "AND" RELIABLY ON SIGNED 16-BIT, BUT WE JUST NEED
    REM TO CHECK IF THE HIGH BIT (MSB) IS SET. SO JUST LOOK AT THE HIGH-BYTE
    REM BY DIVIDING THE VALUE BY 256 (SAME AS SHR >> 8).
    CRC.LOW% = CRC.RESULT / 256
    IF CRC.LOW% AND $80 THEN GOTO HANDLE.HIGH.BIT.CASE

    REM ELSE...
    CRC.RESULT = CRC.RESULT * 2 : REM (CRC << 1)
  HIGH.BIT.WAS.SET:
    IF CRC.RESULT > 32767 THEN CRC.RESULT = CRC.RESULT - 65536
    IF CRC.RESULT < -32768 THEN CRC.RESULT = CRC.RESULT + 65536
    PRINT " ";I;" ";:GOSUB PRINT.CRC.AS.HEX : PRINT : REM PRINT HEX$(CRC.RESULT)
  NEXT

  DATA.IDX% = DATA.IDX% + 1
  GOTO NEXT.BYTE

HANDLE.HIGH.BIT.CASE:
```


[illegible]

CRC16 LOOK UP TABLE (for XModem-CRC or YModem)

To substantially improve the runtime performance of computing CRC, a Lookup Table can be used. This “cost” 512 bytes of RAM, but avoids a bit-shift loop that would otherwise be necessary at runtime.

```
unsigned int CRC_LUT[256] = {
//      HEX DEC Offset
0x0000, // 00 0
0x1021, // 01 1
0x2042, // 02 2
0x3063, // 03 3
0x4084, // 04 4
0x50A5, // 05 5
0x60C6, // 06 6
0x70E7, // 07 7
0x8108, // 08 8
0x9129, // 09 9
0xA14A, // 0A 10
0xB16B, // 0B 11
0xC18C, // 0C 12
0xD1AD, // 0D 13
0xE1CE, // 0E 14
0xF1EF, // 0F 15
0x1231, // 10 16
0x0210, // 11 17
0x3273, // 12 18
0x2252, // 13 19
0x52B5, // 14 20
0x4294, // 15 21
0x72F7, // 16 22
0x62D6, // 17 23
0x9339, // 18 24
0x8318, // 19 25
0xB37B, // 1A 26
0xA35A, // 1B 27
0xD3BD, // 1C 28
0xC39C, // 1D 29
0xF3FF, // 1E 30
0xE3DE, // 1F 31
0x2462, // 20 32
0x3443, // 21 33
0x0420, // 22 34
0x1401, // 23 35
0x64E6, // 24 36
0x74C7, // 25 37
0x44A4, // 26 38
0x5485, // 27 39
0xA56A, // 28 40
0xB54B, // 29 41
0x8528, // 2A 42
0x9509, // 2B 43
0xE5EE, // 2C 44
0xF5CF, // 2D 45
0xC5AC, // 2E 46
0xD58D, // 2F 47
0x3653, // 30 48
0x2672, // 31 49
0x1611, // 32 50
0x0630, // 33 51
0x76D7, // 34 52
0x66F6, // 35 53
0x5695, // 36 54
0x46B4, // 37 55
0xB75B, // 38 56
0xA77A, // 39 57
0x9719, // 3A 58
0x8738, // 3B 59
0xFF7F, // 3C 60
0xEF7E, // 3D 61
0xD79D, // 3E 62
0xC7BC, // 3F 63
0x48C4, // 40 64
0x58E5, // 41 65
0x6886, // 42 66
0x78A7, // 43 67
0x0840, // 44 68
```

0x1861, // 45 69
0x2802, // 46 70
0x3823, // 47 71
0xC9CC, // 48 72
0xD9ED, // 49 73
0xE98E, // 4A 74
0xF9AF, // 4B 75
0x8948, // 4C 76
0x9969, // 4D 77
0xA90A, // 4E 78
0xB92B, // 4F 79
0x5AF5, // 50 80
0x4AD4, // 51 81
0x7AB7, // 52 82
0x6A96, // 53 83
0x1A71, // 54 84
0x0A50, // 55 85
0x3A33, // 56 86
0x2A12, // 57 87
0xDBFD, // 58 88
0xCBDC, // 59 89
0xFBBF, // 5A 90
0xEB9E, // 5B 91
0x9B79, // 5C 92
0x8B58, // 5D 93
0xBB3B, // 5E 94
0xAB1A, // 5F 95
0x6CA6, // 60 96
0x7C87, // 61 97
0x4CE4, // 62 98
0x5CC5, // 63 99
0x2C22, // 64 100
0x3C03, // 65 101
0x0C60, // 66 102
0x1C41, // 67 103
0xEDAE, // 68 104
0xFD8F, // 69 105
0xCDEC, // 6A 106
0xDDCD, // 6B 107
0xAD2A, // 6C 108
0xBD0B, // 6D 109
0x8D68, // 6E 110
0x9D49, // 6F 111
0x7E97, // 70 112
0x6EB6, // 71 113
0x5ED5, // 72 114
0x4EF4, // 73 115
0x3E13, // 74 116
0x2E32, // 75 117
0x1E51, // 76 118
0x0E70, // 77 119
0xFF9F, // 78 120
0xEFFE, // 79 121
0xDFDD, // 7A 122
0xCFFC, // 7B 123
0xBF1B, // 7C 124
0xAF3A, // 7D 125
0x9F59, // 7E 126
0x8F78, // 7F 127
0x9188, // 80 128
0x81A9, // 81 129
0xB1CA, // 82 130
0xA1EB, // 83 131
0xD10C, // 84 132
0xC12D, // 85 133
0xF14E, // 86 134
0xE16F, // 87 135
0x1080, // 88 136
0x00A1, // 89 137
0x30C2, // 8A 138
0x20E3, // 8B 139
0x5004, // 8C 140
0x4025, // 8D 141
0x7046, // 8E 142
0x6067, // 8F 143
0x83B9, // 90 144
0x9398, // 91 145
0xA3FB, // 92 146
0xB3DA, // 93 147
0xC33D, // 94 148
0xD31C, // 95 149
0xE37F, // 96 150

0xF35E, // 97 151
0x02B1, // 98 152
0x1290, // 99 153
0x22F3, // 9A 154
0x32D2, // 9B 155
0x4235, // 9C 156
0x5214, // 9D 157
0x6277, // 9E 158
0x7256, // 9F 159
0xB5EA, // A0 160
0xA5CB, // A1 161
0x95A8, // A2 162
0x8589, // A3 163
0xF56E, // A4 164
0xE54F, // A5 165
0xD52C, // A6 166
0xC50D, // A7 167
0x34E2, // A8 168
0x24C3, // A9 169
0x14A0, // AA 170
0x0481, // AB 171
0x7466, // AC 172
0x6447, // AD 173
0x5424, // AE 174
0x4405, // AF 175
0xA7DB, // B0 176
0xB7FA, // B1 177
0x8799, // B2 178
0x97B8, // B3 179
0xE75F, // B4 180
0xF77E, // B5 181
0xC71D, // B6 182
0xD73C, // B7 183
0x26D3, // B8 184
0x36F2, // B9 185
0x0691, // BA 186
0x16B0, // BB 187
0x6657, // BC 188
0x7676, // BD 189
0x4615, // BE 190
0x5634, // BF 191
0xD94C, // C0 192
0xC96D, // C1 193
0xF90E, // C2 194
0xE92F, // C3 195
0x99C8, // C4 196
0x89E9, // C5 197
0xB98A, // C6 198
0xA9AB, // C7 199
0x5844, // C8 200
0x4865, // C9 201
0x7806, // CA 202
0x6827, // CB 203
0x18C0, // CC 204
0x08E1, // CD 205
0x3882, // CE 206
0x28A3, // CF 207
0xCB7D, // D0 208
0xDB5C, // D1 209
0xEB3F, // D2 210
0xFB1E, // D3 211
0x8BF9, // D4 212
0x9BD8, // D5 213
0xABBB, // D6 214
0xBB9A, // D7 215
0x4A75, // D8 216
0x5A54, // D9 217
0x6A37, // DA 218
0x7A16, // DB 219
0x0AF1, // DC 220
0x1AD0, // DD 221
0x2AB3, // DE 222
0x3A92, // DF 223
0xFD2E, // E0 224
0xED0F, // E1 225
0xDD6C, // E2 226
0xCD4D, // E3 227
0xBDAA, // E4 228
0xAD8B, // E5 229
0x9DE8, // E6 230
0x8DC9, // E7 231
0x7C26, // E8 232

```

0x6C07, // E9 233
0x5C64, // EA 234
0x4C45, // EB 235
0x3CA2, // EC 236
0x2C83, // ED 237
0x1CE0, // EE 238
0x0CC1, // EF 239
0xEF1F, // F0 240
0xFF3E, // F1 241
0xCF5D, // F2 242
0xDF7C, // F3 243
0xAF9B, // F4 244
0xBFBA, // F5 245
0x8FD9, // F6 246
0x9FF8, // F7 247
0x6E17, // F8 248
0x7E36, // F9 249
0x4E55, // FA 250
0x5E74, // FB 251
0x2E93, // FC 252
0x3EB2, // FD 253
0x0ED1, // FE 254
0x1EF0, // FF 255
};

```

This table is computed by effectively running the CRC bit-shift loop itself:

```

void init_CRC_table() {
    for (uint16_t j = 0; j < 256; j++) {
        uint16_t crc = j << 8;
        for (uint8_t i = 0; i < 8; i++) {
            if (crc & 0x8000) {
                crc = (crc << 1) ^ 0x1021; // XModem CRC polynomial "magic number"
            } else {
                crc <<= 1;
            }
        }
        CRC_LUT[j] = crc;
    }
}

```

To use this Lookup Table, the CRC16 result is now computed as follows:

```

CRC_RESULT = (CRC_RESULT << 8) ^ CRC_LUT[ ((CRC_RESULT >> 8) ^ SERIAL_IN) & 0xFF ];

```

Where CRC_RESULT is initialized as 0, SERIAL_IN is the received UART byte during the data-portion of each block.

THE YMODEM BATCH PROTOCOL SEQUENCE

<http://www.blunk-electronic.de/train-z/pdf/xymodem.pdf>

```
#define ASCII_NULL 0
#define ASCII_SOH 1 // Start of Header [ 128 mode ]
#define ASCII_STX 2 // Start of Text [ 1024 mode ]
#define ASCII_EOT 4 // End of Transmission
#define ASCII_ACK 6 // Acknowledge
#define ASCII_SPACE 32 // " "
#define ASCII_NAK 21 // Negative Acknowledge (or checksum)
#define ASCII_C 67 // Upper case "C"
#define CPM_EOF 0x1A // CTRL-Z, for CP/M systems
```

NOTE: XModem-CRC also supports being initiated with "C" but does not have the header block. Regular XModem is initiated with a NAK byte, which then uses the CHECKSUM 8-bit approach (of just summing the byte-value of all the block content). Technically a single protocol source could adapt to either protocol. But for purposes here, we focus just on YModem.

The following is a cleaned up "modernized" version of the protocol sequence described in the "True YModem" document from 1988. This is a batch of two small files:

(uploader)	(download)
SENDER	RECEIVER
(select files, initialize "upload")	(load/initialize downloader software)
	"C"
SOH 00 FF	(null terminates)
FILE_NAME \0	(decimal) " " (SPACE) terminated
FILE_LENGTH SPACE	(date/time octal, unix System Time seconds)
FILE_DATE SPACE	("0" if non-unix, otherwise octal)
FILE_MODE SPACE	
FILE_SN SPACE	
HEADER_PAD[128]	(the last header entry present is null terminates)
	(128 - string length of header content)
	(if filename + header content is beyond 128 bytes, this is padded to 1024 and header uses STX)
CRC_HI CRC_LO	(check CRC - NAK if not matched)
	ACK
	"C"
SOH 01 FE	(HDR ~HDR, header marker followed by "inverse" header marker)
BLOCK_DATA[128]	
CRC_HI CRC_LO	(check CRC - NAK if not matched)
	ACK
SOH 02 FD	
BLOCK_DATA[128]	
CRC_HI CRC_LO	(check CRC - NAK if not matched)

```

                                ACK

SOH 03 FC
BLOCK_DATA[100]
CPMEOF[28]

                                (in this last "partially filled block", the receiver knows
                                the expected actual byte-length of the file; probably for compatibility
                                with CP/M or backwards compatible with XModem, data-block entry past
                                that specified length are CP/M EOF hex 1A)

CRC_HI CRC_LO

                                (check CRC - NAK if not matched)
                                ACK

EOT      (first End of Transmission marker)

                                NAK      ("fake NAK", initial EOT indicator)

EOT      (second End of Transmission marker)

                                ACK      (actual ACK to the second EOT indicator)
                                "C"    (restart the protocol, CRC continue next file)

SOH 00 FF
FILENAME \0
HEADER_PAD[128]
CRC_HI CRC_LO

                                (check CRC - NAK if not matched)
                                ACK
                                "C"

SOH 01 FE
BLOCK_DATA[100]
CPMEOF[28]      (populate whatever past expected filelength with CPM EOF)
                  (this content doesn't really matter, since ideally the receiver will
                  never be writing it - unless it doesn't do its own filelength crop)

CRC_HI CRC_LO

                                (check CRC - NAK if not matched)
                                ACK

EOT

                                NAK      ("fake NAK", initial EOT indicator)

EOT

                                ACK      (actual ACK to the second EOT indicator)
                                "C"

SOH 00 FF
HEADER_PAD[128]
00 00      (CRC of no-filename, no-length naturally computed as "00 00")
                                ACK      (multiple ACK is also ok, up to 10x)

```

YMODEM-G PROTOCOL SEQUENCE

<http://www.blunk-electronic.de/train-z/pdf/xymodem.pdf>

YModem-G can function with just having proper hardware CTS/RTS handling. Basically this “push/pull”s the bytes straight through the FIFO as quickly as possible.

(uploader)	(download)
SENDER	RECEIVER
(select files, initialize “upload”)	(load/initialize downloader software)
	“G”
SOH 00 FF	
FILENAME \0	
HEADER_PAD[128]	
CRC_HI CRC_LO	
	“G” [or CAN CAN to abort]
SOH 01 FE	
BLOCK_DATA[128]	
CRC_HI CRC_LO	
	(check CRC) [CAN CAN to abort]
SOH 02 FD	
BLOCK_DATA[128]	
CRC_HI CRC_LO	
	(check CRC) [CAN CAN to abort]
SOH 03 FC	
BLOCK_DATA[100]	
CPMEOF[28] (or whatever the remainder of the 128-byte block is)	
CRC_HI CRC_LO	
	(check CRC) [CAN CAN to abort]
EOT	
	ACK
	“G”
SOH 00 FF	
FILENAME \0	
HEADER_PAD[128]	
CRC_HI CRC_LO	
	(check CRC)
	“G”
SOH 01 FE	
BLOCK_DATA[100]	
CPMEOF[28]	
CRC_HI CRC_LO	
EOT	
	(check CRC)
	ACK
	“C”
SOH 00 FF	
HEADER_PAD[128]	
00 00	
	ACK (or ACK ACK, really up to 10x ACKs is ok)

End.